

Project Proposal — Forecasting One Piece TCG Swiss Matches

Kiana Kiser · MLOps HS26 · I.BA_MLOPS · Repository (public): github.com/kianakiser/optcg-match-forecast

1 Problem statement

What and for whom. For every Swiss match at an online One Piece TCG tournament, predict the probability the seat-1 player wins, from both registered decklists and both pilots' results at strictly earlier events. For competitive players and deck-builders.

Horizon. Emitted when a round's pairings publish — both 50-card lists registered, no card played — and resolved when the platform reports the winner. That reported winner is the label: it is not computable from any feature. One match, not a time window.

Scope. Online Swiss rounds, predominantly best-of-one, at 32+ entrant events on play.limitlesstcg.com; top cut, byes, ties and unresolvable decklists excluded — 28,107 matches as of 2026-09-18, 82.1% of them from one recurring series, so claims cover online play only.

Success criterion. Baseline: a coin flip — Brier 0.2500, log loss 0.6931 — because the label is balanced by construction: seat 1 wins 50.5% of 29,004 decided pairings in the cached corpus (Wilson 95% CI 50.0–51.1%), leaving no majority-class rule and no rare positive class. Success = pooled over ≥ 6 held-out 28-day windows **split by event date, never by row** ($\geq 8,000$ matches): Brier ≤ 0.2490 with the skill interval (0.2500 – Brier, whole-event bootstrap) excluding zero, and calibration gaps ≤ 3 points in every 5-point favourite bucket with $n \geq 250$. Reproducing the existing archetype model out-of-time gives 0.2465; 0.2490 is the bar every six-window stretch in 14 months clears.

2 Originality & motivation

[Kiana writes this one — “why this problem, and why you” is the graded question and it has to be yours. Raw material: you already built and run the data pipeline this draws on; the modelling question is separating deck strength from pilot skill, which is a real confounder rather than a leaderboard; and competitive TCG match forecasting appears on neither the mlops-lab.ch showcase nor the KTH ID2223 lists. Check both explicitly and say so.]

3 Data source & features

Matches come from Limitless (play.limitlesstcg.com) through its public JSON API — `/tournaments`, `/tournaments/{id}/standings`, `/tournaments/{id}/pairings` — no auth, no scraping. GitHub Actions ingests daily; events are immutable once finished, so each is fetched once and appears the morning after it ends. Today that is 28,107 decided Swiss matches over 193 events and 11,890 entries, 11,886 of them with a complete 50-card decklist, growing ≈ 570 matches and 3.5 events weekly.

The label is `pairings[].winner`, the username the platform records from the reported result; it is not computable from either decklist. It is balanced — seat 1 wins 50.5% (95% CI 49.9–51.1%, $n = 27,506$) — so there is no rare positive class; scarcity sits in the archetype tail, where 13.0% of held-out deck-sides use a leader unseen in training.

Features, per side and differenced: `leader_id`, `leader_life`, `leader_power`, `leader_color_count`; deck aggregates `counter_2k_copies`, `avg_cost`, `high_curve_copies`, `big_body_copies`, `event_card_copies`, `distinct_cards`; and point-in-time `player_prior_winrate` and `archetype_prior_winrate`, shrunk toward 0.5 over strictly earlier events.

Leakage. `placing`, `record` and `drop` are outcomes of the predicted event and are dropped physically at ingest. `placing` is null on 43.0% of 12,166 entries and every such row carries `drop`, so filtering `placing IS NOT NULL` would condition the sample on finishing the event. Splits are by event date, not by row: matches from one event share decks and pilots. The ingest persists each event's date from the tournament index, which the pairings payload omits.

4 System design

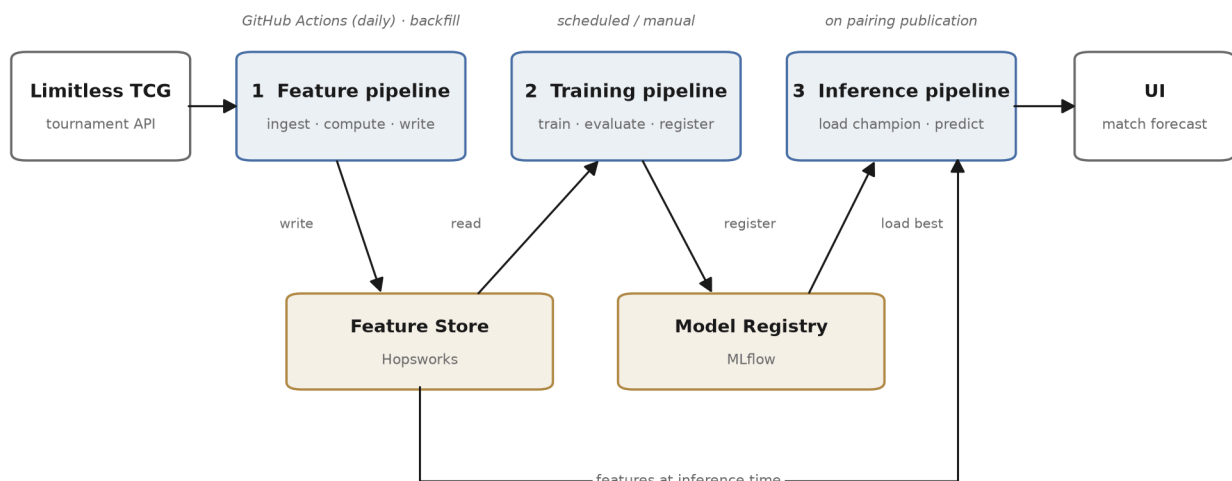


Figure 1: The three FTI pipelines. They are decoupled — none calls another; they meet only at the feature store and the model registry.

Core (ships first). A GitHub Actions job runs daily at 06:07 UTC, matching the source's ≈ 3.5 events a week. It pulls the Limitless API, drops `placing`, `record` and `drop` physically at the ingest boundary, persists each event's date from the tournament index — the pairings payload carries none, so without it no out-of-time split is possible — lands raw JSON in GCS and writes features to Hopsworks. Training reads the feature view, splits by event date, logs to MLflow and registers the winner as champion. Cloud Run loads `models:/...@champion` and returns a win probability for pairings the Actions job posts to it, logging predictions for scoring; serving never calls the provider.

Stack. *Hopsworks*: point-in-time joins, so player history cannot include the event being predicted. *MLflow*: promotion moves an alias instead of redeploying. *GitHub Actions*: schedules beside code and CI, and sole holder of the provider client — throttled to the published 50 requests per 5 minutes, finished events cached write-once. *Cloud Run*: container deploy, scales to zero between events. *GCS*: immutable landing zone, so features rebuild without re-fetching.

Optional stretch, outside the core: decklist drift monitoring, a second feed to dilute the $\approx 82\%$ -of-matches single-organiser concentration, and a calibration dashboard.

The repository is public: github.com/kianakiser/optcg-match-forecast.